🏛️ ComplianceGuard — Architecture CrewAI Complète

> Agent IA de Conformité Juridique Tunisienne avec GraphRAG

---

1. Vue d'ensemble des Sources de Données

Après analyse des sites officiels, voici ce qui est accessible/scrappable :

Source	Format	Accessibilité	Stratégie d'ingestion
legislation.tn	HTML statique (textes de lois)	Scraping possible	`ScrapeWebsiteTool` + parsing
inpdp.tn/Texte.html	HTML + PDF directs	✅ PDFs téléchargeables	`PDFSearchTool` sur les PDFs
inpdp.tn PDFs	PDF (loi 2004-63, décrets)	✅ URL directes publiques	`PDFSearchTool`
JORT	PDF par numéro	Scraping structuré	`ScrapeWebsiteTool` + parser
APII	HTML / formulaires	SSL expiré — copie manuelle	Fichiers locaux
Startup Act	PDF officiel	Téléchargement manuel	`PDFSearchTool`

Constat clé : Les PDFs de l'INPDP ont des URLs directes publiques, donc tu peux les ingérer directement dans ton vector store.

---

2. Architecture Générale de l'Agent

```
┌──────────────────────────────────────────────────────────────────────────┐
│                          COMPLIANCEGUARD CREW                            │
│                                                                          │
│  ┌──────────────────┐  ┌──────────────────┐  ┌──────────────────┐      │
│  │ Legal            │  │ Compliance       │  │ Document         │      │
│  │ Retriever        │→ │ Analyst          │→ │ Generator        │      │
│  │ Agent            │  │ Agent            │  │ Agent            │      │
│  └──────────┬───────┘  └──────────┬───────┘  └──────────┬───────┘      │
│             │                     │                     │              │
│             ▼                     ▼                     ▼              │
```

```
│   │GraphRAG │     │Rules   │     │Template  │    │
│   │Tool    │     │Engine  │     │Tool      │    │
│   │(Neo4j) │     │Tool    │     │          │    │
│   └────────┘     └────────┘     └──────────┘    │
│                                 │
│          JORT Monitor Agent (cron)              │
└─────────────────────────────────────────────────┘
```

---

3. Les Tools CrewAI à Utiliser

3.1 Tools Natifs CrewAI (crewai-tools)

```python
from crewai_tools import (
    PDFSearchTool,
    ScrapeWebsiteTool,
    RagTool,
    FileWriterTool,
    FileReadTool,
    DirectoryReadTool,
)
```

📄 PDFSearchTool — Pour les lois en PDF

```python
```

```python
from crewai_tools import PDFSearchTool

# Loi organique 2004-63 (INPDP)
inpdp_law_tool = PDFSearchTool(
    pdf="https://www.inpdp.tn/ressources/loi_2004.pdf",
    config={
        "llm": {"provider": "openai", "config": {"model": "gpt-4o-mini"}},
        "embedder": {
            "provider": "openai",
            "config": {"model": "text-embedding-3-small"}
        }
    }
)

# Décret 2007-3004 (procédures de déclaration)
decret_tool = PDFSearchTool(
    pdf="https://www.inpdp.tn/ressources/decret_3004.pdf"
)

# Startup Act
startup_act_tool = PDFSearchTool(
    pdf="./data/startup_act.pdf"  # téléchargé localement
)
```

🌐 ScrapeWebsiteTool — Pour legislation.tn

```python
from crewai_tools import ScrapeWebsiteTool

legislation_scraper = ScrapeWebsiteTool(
    website_url="http://www.legislation.tn"
)
# Usage dynamique (donner URL à l'agent)
dynamic_scraper = ScrapeWebsiteTool()
```

📚 RagTool — Base juridique unifiée

```python
```

```python
from crewai_tools import RagTool

# Créer une base RAG consolidée
legal_rag = RagTool()
legal_rag.add(data_type="file", path="./data/code_commerce.pdf")
legal_rag.add(data_type="file", path="./data/code_societes.pdf")
legal_rag.add(data_type="file", path="./data/loi_2004_63.pdf")
legal_rag.add(data_type="web_page", url="https://www.inpdp.tn/Texte.html")
```

✍️ FileWriterTool — Pour générer des documents

```python
from crewai_tools import FileWriterTool
doc_writer = FileWriterTool()
# L'agent écrit CGU, CGV, Politique de confidentialité
```

3.2 Custom Tools à Créer

🔧 Tool 1 : Legal Rules Engine

```python
```

```python
from crewai.tools import BaseTool
from pydantic import BaseModel, Field
from typing import Type
import json

class LegalRulesInput(BaseModel):
    activity_type: str = Field(description="Type d'activité: ecommerce, sarl, sa, suarl, fintech, etc.")
    sub_domain: str = Field(description="Sous-domaine: creation, fiscalite, donnees, paiement")

class LegalRulesEngineTool(BaseTool):
    name: str = "Legal Rules Engine"
    description: str = (
        "Associe un type d'activité aux obligations légales tunisiennes correspondantes. "
        "Retourne les lois applicables, les obligations et les risques de non-conformité."
    )
    args_schema: Type[BaseModel] = LegalRulesInput

    RULES_DB = {
        "ecommerce": {
            "lois": ["Loi 2000-83 sur le commerce électronique", "Loi 2004-63 INPDP"],
            "obligations": [
                "Déclaration INPDP obligatoire",
                "CGU/CGV conformes",
                "Mentions légales",
                "Sécurisation paiements (BCT)",
            ],
            "risques": ["Amende jusqu'à 50 000 DT si pas de déclaration INPDP"]
        },
        "sarl": {
            "lois": ["Code des Sociétés Commerciales Art. 90-140"],
            "obligations": [
                "Capital minimum 1000 DT",
                "Registre de commerce (RNE)",
                "Publication au JORT",
                "Déclaration fiscale initiale"
            ],
            "risques": ["Nullité de la société si formalités non respectées"]
        },
        "startup_act": {
            "lois": ["Décret-loi 2018-20 Startup Act"],
            "obligations": [
                "Label Startup (via startup.gov.tn)",
                "Critères: <8 ans, innovante, <100 employés"
```

```
        ],
        "avantages": [
            "Exonération IS 8 ans",
            "Congé entrepreneuriat",
            "Facilités de change"
        ]
    },
    "donnees_personnelles": {
        "lois": ["Loi organique 2004-63", "Décret 2007-3004"],
        "obligations": [
            "Déclaration préalable à l'INPDP",
            "Autorisation pour données sensibles",
            "Nomination DPO recommandé",
            "Politique de confidentialité"
        ]
    }
}

def _run(self, activity_type: str, sub_domain: str) -> str:
    key = activity_type.lower()
    rules = self.RULES_DB.get(key, {})
    if not rules:
        return f"Pas de règles prédéfinies pour '{activity_type}'. Rechercher dans la base légale."
    return json.dumps(rules, ensure_ascii=False, indent=2)
```

🔧 Tool 2 : JORT Monitor Tool

```python
```

```python
import requests
from bs4 import BeautifulSoup
from crewai.tools import BaseTool

class JORTMonitorTool(BaseTool):
    name: str = "JORT Monitor"
    description: str = (
        "Surveille le Journal Officiel de la République Tunisienne (JORT) "
        "pour détecter les nouvelles lois et décrets pertinents pour les entrepreneurs."
    )

    def _run(self, keyword: str = "société") -> str:
        # Scraping du JORT (adapter selon la structure réelle)
        url = f"http://www.iort.gov.tn/WD120AWP/WD120Awp.exe/CONNECT/JORTConnect"
        try:
            # Recherche par mot-clé
            results = []
            # TODO: adapter selon l'API/structure JORT
            return f"Recherche JORT pour '{keyword}': Connecter à iort.gov.tn"
        except Exception as e:
            return f"Erreur JORT: {str(e)}"
```

🔧 Tool 3 : Document Template Generator

```python
```

```
from crewai.tools import BaseTool
from pydantic import BaseModel, Field
from typing import Type

class DocGenInput(BaseModel):
    doc_type: str = Field(description="Type: cgu, cgv, privacy_policy, mentions_legales")
    company_name: str = Field(description="Nom de la société")
    activity: str = Field(description="Description de l'activité")
    email: str = Field(description="Email de contact")

class DocumentGeneratorTool(BaseTool):
    name: str = "Legal Document Generator"
    description: str = (
        "Génère des documents juridiques tunisiens conformes: CGU, CGV, "
        "Politique de confidentialité, Mentions légales."
    )
    args_schema: Type[BaseModel] = DocGenInput

    def _run(self, doc_type: str, company_name: str, activity: str, email: str) -> str:
        templates = {
            "privacy_policy": f"""
POLITIQUE DE CONFIDENTIALITÉ — {company_name}
Conformément à la Loi Organique n° 2004-63 du 27 juillet 2004

1. RESPONSABLE DU TRAITEMENT
{company_name} — Contact: {email}

2. DONNÉES COLLECTÉES
Dans le cadre de l'activité {activity}, nous collectons...

3. FINALITÉS DU TRAITEMENT
Les données sont traitées pour...

4. DÉCLARATION INPDP
Ce traitement a été déclaré/autorisé par l'INPDP.

5. DROITS DES PERSONNES
Conformément à l'article 8 de la loi 2004-63, vous disposez d'un droit d'accès...
""",
            "mentions_legales": f"""
MENTIONS LÉGALES — {company_name}
Conformément à la Loi n° 2000-83 sur le commerce électronique
```

```
Éditeur: {company_name}
Activité: {activity}
Contact: {email}
Registre de Commerce: [RC_NUMBER]
Matricule Fiscal: [MF_NUMBER]
"""
    }
    return templates.get(doc_type, f"Template '{doc_type}' non disponible.")
```

🔧 Tool 4 : Compliance Checklist Generator

```python
```

```python
class ComplianceChecklistTool(BaseTool):
    name: str = "Compliance Checklist Generator"
    description: str = "Génère une checklist personnalisée de conformité selon le type de projet."

    def _run(self, project_type: str, has_personal_data: bool = True) -> str:
        checklist = {
            "🏢 CRÉATION": [
                "☐ Rédaction des statuts",
                "☐ Dépôt au Greffe du Tribunal",
                "☐ Inscription au RNE (registre.tn)",
                "☐ Publication au JORT",
                "☐ Obtention de la carte d'immatriculation fiscale",
            ],
            "📊 FISCAL": [
                "☐ Déclaration d'existence à la recette des finances",
                "☐ Choix régime TVA (si applicable)",
                "☐ Acomptes provisionnels IS",
            ],
            "🔒 DONNÉES PERSONNELLES": [] if not has_personal_data else [
                "☐ Inventaire des traitements",
                "☐ Déclaration INPDP (www.inpdp.tn)",
                "☐ Rédaction politique de confidentialité",
                "☐ Mise en place procédure droits d'accès",
            ],
            "🛒 E-COMMERCE (si applicable)": [
                "☐ CGU/CGV conformes loi 2000-83",
                "☐ Mentions légales sur le site",
                "☐ Sécurisation paiements (agrément BCT)",
            ]
        }
        result = f"CHECKLIST CONFORMITÉ — {project_type}\n\n"
        for section, items in checklist.items():
            result += f"{section}\n" + "\n".join(items) + "\n\n"
        return result
```

---

4. Définition des Agents CrewAI

```python
```

```python
from crewai import Agent, Task, Crew, Process
from langchain_openai import ChatOpenAI

llm = ChatOpenAI(model="gpt-4o", temperature=0.1)

# Agent 1 : Récupération juridique
legal_retriever = Agent(
    role="Expert en Droit Tunisien",
    goal="Trouver les textes de lois tunisiens applicables à la situation décrite",
    backstory="""Tu es un juriste spécialisé en droit des affaires tunisien avec
    15 ans d'expérience. Tu maîtrises le Code des Sociétés, le Code de Commerce,
    la Loi 2004-63 INPDP, le Startup Act et toutes les réglementations sectorielles.""",
    tools=[legal_rag, inpdp_law_tool, dynamic_scraper, startup_act_tool],
    llm=llm,
    verbose=True,
    memory=True,
)

# Agent 2 : Analyste de conformité
compliance_analyst = Agent(
    role="Analyste de Conformité Réglementaire",
    goal="Analyser la conformité d'un projet et identifier tous les risques réglementaires",
    backstory="""Tu es expert en conformité pour startups tunisiennes.
    Tu identifies les obligations légales, les risques et tu priorises les actions.""",
    tools=[legal_rules_tool, compliance_checklist_tool],
    llm=llm,
    verbose=True,
)

# Agent 3 : Générateur de documents
doc_generator = Agent(
    role="Rédacteur Juridique",
    goal="Générer des documents juridiques conformes au droit tunisien",
    backstory="""Tu rédiges des documents juridiques clairs et conformes pour startups tunisiennes:
    CGU, CGV, Politique de confidentialité, Mentions légales.""",
    tools=[doc_generator_tool, doc_writer],
    llm=llm,
    verbose=True,
)

# Agent 4 : Veilleur JORT
jort_monitor = Agent(
    role="Veilleur Réglementaire",
```

```
    goal="Surveiller les nouvelles réglementations et alerter sur les changements",
    backstory="Tu surveilles en permanence le JORT et les publications officielles tunisiennes.",
    tools=[jort_monitor_tool, dynamic_scraper],
    llm=llm,
    verbose=True,
)
```

---

5. Intégration GraphRAG

Pourquoi GraphRAG pour ComplianceGuard ?

Le GraphRAG est **idéal** pour la conformité juridique car les lois tunisiennes forment un **réseau de relations** :

```
Loi 2004-63 ─────── modifie ──────▶ Décret 2007-3004
    │                       │
    └──────── concerne ──────▶ SARL      └──────── procédure ──▶ Formulaire INPDP
    │
    └──────── référence ──────▶ Constitution Art.24
    │
    └──────── sanctionne ──────▶ Amende 50.000 DT
```

Le RAG classique (vectoriel) ne capture pas ces **relations**. GraphRAG les exploite.

Stack Recommandée

```
Neo4j (graph DB) + LlamaIndex GraphRAG + CrewAI Custom Tool
```

5.1 Installation

```bash
pip install crewai crewai-tools
pip install llama-index llama-index-graph-stores-neo4j
pip install neo4j
pip install langchain-community
```

5.2 Construction du Knowledge Graph Juridique

```python
```

```python
from llama_index.core import KnowledgeGraphIndex, SimpleDirectoryReader
from llama_index.graph_stores.neo4j import Neo4jGraphStore
from llama_index.core import StorageContext
from llama_index.llms.openai import OpenAI

# 1. Configuration Neo4j
graph_store = Neo4jGraphStore(
    username="neo4j",
    password="your_password",
    url="bolt://localhost:7687",
    database="complianceguard"
)

storage_context = StorageContext.from_defaults(graph_store=graph_store)

# 2. Chargement des documents juridiques
documents = SimpleDirectoryReader("./data/legal_docs").load_data()
# ./data/legal_docs/ contient:
#   - loi_2004_63.pdf
#   - code_societes.pdf
#   - code_commerce.pdf
#   - startup_act.pdf
#   - decret_3004.pdf

# 3. Construction de l'index avec extraction de triplets
llm = OpenAI(model="gpt-4o", temperature=0)
index = KnowledgeGraphIndex.from_documents(
    documents,
    storage_context=storage_context,
    max_triplets_per_chunk=10,
    llm=llm,
    include_embeddings=True,  # Hybrid: Graph + Vector
)

# 4. Query engine hybride (Graph + Vector RAG)
query_engine = index.as_query_engine(
    include_text=True,
    response_mode="tree_summarize",
    embedding_mode="hybrid",  # Utilise à la fois le graphe et les embeddings
    similarity_top_k=5,
)
```

5.3 Custom GraphRAG Tool pour CrewAI

```python
from crewai.tools import BaseTool
from pydantic import BaseModel, Field
from typing import Type

class GraphRAGInput(BaseModel):
    query: str = Field(description="Question juridique en français ou arabe")

class GraphRAGLegalTool(BaseTool):
    name: str = "GraphRAG Legal Knowledge Base"
    description: str = (
        "Recherche dans la base de connaissance juridique tunisienne structurée en graphe. "
        "Trouve les relations entre lois, obligations, sanctions et procédures. "
        "Utilise pour des questions complexes impliquant plusieurs textes de lois."
    )
    args_schema: Type[BaseModel] = GraphRAGInput
    query_engine: object = None  # injecté à l'init

    def _run(self, query: str) -> str:
        if self.query_engine is None:
            return "GraphRAG non initialisé."
        response = self.query_engine.query(query)
        return str(response)

# Utilisation
graph_rag_tool = GraphRAGLegalTool(query_engine=query_engine)
```

5.4 Schéma des Entités dans Neo4j

```cypher
```

```
// Modèle de graphe juridique tunisien
CREATE (l1:Loi {id: "2004-63", titre: "Loi protection données", annee: 2004})
CREATE (d1:Decret {id: "2007-3004", titre: "Déclaration INPDP", annee: 2007})
CREATE (o1:Obligation {nom: "Déclaration_préalable", delai: "avant_traitement"})
CREATE (s1:Sanction {montant: "50000_DT", type: "amende"})
CREATE (e1:Entite {type: "SARL"})
CREATE (p1:Procedure {nom: "Formulaire_INPDP", url: "inpdp.tn/Formulaires.html"})

// Relations
CREATE (l1)-[:MODIFIE_PAR]->(d1)
CREATE (l1)-[:IMPOSE]->(o1)
CREATE (o1)-[:CONCERNE]->(e1)
CREATE (l1)-[:SANCTIONNE_PAR]->(s1)
CREATE (o1)-[:SUIT_PROCEDURE]->(p1)
```

5.5 Visualisation du Graphe

```python
# Visualiser dans Neo4j Browser avec Cypher:
# MATCH (n)-[r]->(m) RETURN n,r,m LIMIT 100
```

---

6. Architecture Finale Complète

```python
```

```python
# crew.py
from crewai import Crew, Process

compliance_crew = Crew(
    agents=[legal_retriever, compliance_analyst, doc_generator],
    tasks=[
        Task(
            description="""
            L'utilisateur décrit son projet: {project_description}
            1. Identifier le type d'activité et la forme juridique
            2. Rechercher tous les textes légaux applicables
            3. Utiliser GraphRAG pour les relations entre lois
            """,
            expected_output="Liste des textes légaux applicables avec articles précis",
            agent=legal_retriever,
        ),
        Task(
            description="""
            Sur base des textes trouvés, analyser la conformité:
            1. Obligations immédiates (J+0)
            2. Obligations à 30 jours
            3. Obligations permanentes
            4. Générer la checklist personnalisée
            """,
            expected_output="Rapport de conformité structuré avec checklist",
            agent=compliance_analyst,
        ),
        Task(
            description="""
            Générer les documents juridiques nécessaires:
            - Politique de confidentialité (si données personnelles)
            - Mentions légales (si site web)
            - CGU/CGV (si e-commerce)
            Projet: {project_description}
            Société: {company_name}
            """,
            expected_output="Documents juridiques prêts à l'emploi",
            agent=doc_generator,
        ),
    ],
    process=Process.sequential,
    memory=True,
    verbose=True,
```

```
    output_log_file="compliance_logs.txt"
)

# Lancement
result = compliance_crew.kickoff(inputs={
    "project_description": "Je veux créer une SARL e-commerce vendant des produits artisanaux",
    "company_name": "ArtTunisie SARL",
    "email": "contact@arttunisie.tn"
})
```

---

7. Environnement de Test

7.1 Setup du Projet

```bash
# Structure du projet
complianceguard/
├── data/
│   ├── legal_docs/        # PDFs des lois
│   ├── templates/         # Templates de documents
│   └── rules/             # rules_db.json
├── tools/
│   ├── __init__.py
│   ├── legal_rules.py
│   ├── jort_monitor.py
│   ├── doc_generator.py
│   ├── compliance_checklist.py
│   └── graphrag_tool.py
├── agents/
│   ├── legal_retriever.py
│   ├── compliance_analyst.py
│   └── doc_generator.py
├── tests/
│   ├── test_tools.py
│   ├── test_agents.py
│   └── test_scenarios.py
├── crew.py
├── main.py
├── .env
└── pyproject.toml
```

```bash
# Installation
pip install crewai crewai-tools
pip install llama-index llama-index-graph-stores-neo4j
pip install pytest pytest-asyncio
pip install python-dotenv

# .env
OPENAI_API_KEY=sk-...
NEO4J_URI=bolt://localhost:7687
NEO4J_PASSWORD=...
SERPER_API_KEY=...  # optionnel pour recherche web
```

7.2 Neo4j Local (Docker)

```bash
docker run \
  --name complianceguard-neo4j \
  -p7474:7474 -p7687:7687 \
  -e NEO4J_AUTH=neo4j/password123 \
  -e NEO4J_PLUGINS='["apoc", "graph-data-science"]' \
  neo4j:5.15
# Interface: http://localhost:7474
```

7.3 Tests Unitaires des Tools

```python
```

```python
# tests/test_tools.py
import pytest
from tools.legal_rules import LegalRulesEngineTool
from tools.compliance_checklist import ComplianceChecklistTool
from tools.doc_generator import DocumentGeneratorTool

def test_ecommerce_rules():
    tool = LegalRulesEngineTool()
    result = tool._run(activity_type="ecommerce", sub_domain="donnees")
    assert "INPDP" in result
    assert "2004-63" in result

def test_sarl_rules():
    tool = LegalRulesEngineTool()
    result = tool._run(activity_type="sarl", sub_domain="creation")
    assert "1000 DT" in result
    assert "RNE" in result

def test_compliance_checklist():
    tool = ComplianceChecklistTool()
    result = tool._run(project_type="SARL E-commerce", has_personal_data=True)
    assert "INPDP" in result
    assert "RNE" in result

def test_privacy_policy_generation():
    tool = DocumentGeneratorTool()
    result = tool._run(
        doc_type="privacy_policy",
        company_name="TestCo SARL",
        activity="vente en ligne",
        email="test@testco.tn"
    )
    assert "2004-63" in result
    assert "INPDP" in result
```

7.4 Tests des Scénarios Complets

```python
```

```python
# tests/test_scenarios.py
import pytest
from crew import compliance_crew

# Scénarios de test basés sur des cas réels tunisiens
SCENARIOS = [
    {
        "name": "Startup E-commerce",
        "input": {
            "project_description": "Application mobile e-commerce vente vêtements, paiement en ligne, collecte emails clients
            "company_name": "FashionTN SUARL",
            "email": "hello@fashiontn.tn"
        },
        "expected_mentions": ["INPDP", "déclaration", "BCT", "2000-83", "CGU"],
    },
    {
        "name": "Startup Act candidate",
        "input": {
            "project_description": "Solution SaaS B2B pour la gestion RH, 3 associés, 2 ans d'existence",
            "company_name": "HRTech SARL",
            "email": "ceo@hrtech.tn"
        },
        "expected_mentions": ["Startup Act", "label", "exonération", "2018-20"],
    },
    {
        "name": "Fintech",
        "input": {
            "project_description": "Application de paiement mobile et transfert d'argent",
            "company_name": "PayTN SA",
            "email": "contact@paytn.tn"
        },
        "expected_mentions": ["BCT", "autorisation", "Banque Centrale"],
    }
]

@pytest.mark.parametrize("scenario", SCENARIOS)
def test_compliance_scenario(scenario):
    result = compliance_crew.kickoff(inputs=scenario["input"])
    output_text = str(result)
    for keyword in scenario["expected_mentions"]:
        assert keyword in output_text, f"'{keyword}' manquant dans le scénario '{scenario['name']}'"
```

7.5 Interface de Test Simple (CLI)

```python
# main.py
from crew import compliance_crew

def main():
    print("🇹🇳 ComplianceGuard — Agent de Conformité Juridique Tunisienne")
    print("=" * 60)

    project = input("📝 Décrivez votre projet: ")
    company = input("🏢 Nom de votre société: ")
    email = input("📧 Email de contact: ")

    print("\n⚙️ Analyse en cours...\n")

    result = compliance_crew.kickoff(inputs={
        "project_description": project,
        "company_name": company,
        "email": email
    })

    print("\n" + "=" * 60)
    print("✅ RAPPORT DE CONFORMITÉ")
    print("=" * 60)
    print(result)

if __name__ == "__main__":
    main()
```

7.6 Interface Web avec Chainlit (Recommandé)

```bash
pip install chainlit
```

```python
```

```
# app.py
import chainlit as cl
from crew import compliance_crew

@cl.on_message
async def main(message: cl.Message):
    await cl.Message(content="⚙️ Analyse de conformité en cours...").send()

    result = compliance_crew.kickoff(inputs={
        "project_description": message.content,
        "company_name": "Votre Société",
        "email": "contact@email.tn"
    })

    await cl.Message(content=f"✅ **Rapport de Conformité**\n\n{result}").send()
```

```bash
chainlit run app.py
# Ouvre http://localhost:8000
```

---

8. Récapitulatif des Décisions Techniques

Composant	Choix	Justification
Framework agent	**CrewAI**	Multi-agents, orchestration native
RAG vectoriel	**RagTool + FAISS**	Rapide, simple à déployer
GraphRAG	**Neo4j + LlamaIndex**	Relations inter-lois, navigation contextuelle
Embeddings	**text-embedding-3-small**	Multilingue FR/AR, économique
LLM	**GPT-4o-mini** (prod) / **GPT-4o** (analyse complexe)	Rapport qualité/coût
Documents	**PDFSearchTool**	PDFs INPDP accessibles publiquement
Scraping	**ScrapeWebsiteTool**	legislation.tn scrappable
Tests	**pytest + Chainlit**	Tests auto + UI manuelle
Monitoring	**JORT custom tool**	Veille réglementaire tunisienne

9. Roadmap de Développement

Semaine 1: Ingestion des données

✓ Télécharger tous les PDFs INPDP

✓ Scraper legislation.tn (Code Commerce, Sociétés)

✓ Construire le graphe Neo4j

Semaine 2: Tools custom

✓ Legal Rules Engine (rules_db.json complet)

✓ Document Generator (templates FR/AR)

✓ Compliance Checklist Tool

Semaine 3: Agents & Crew

✓ Intégrer tous les tools

✓ Tester les 3 scénarios principaux

Semaine 4: Interface & Production

✓ Chainlit UI

✓ JORT Monitor (cron quotidien)

✓ Déploiement (Railway / Render)